

Meeting 18/03/2026

Summary

Action Items & Next Steps

- ☐ Students to develop a high-level design and architecture proposal for the bug triaging system ▶ ▶
- ☐ Students to create a POC to validate the proposed design before proceeding to detailed design stage ▶
- ☐ Students to develop a project timeline and plan for task distribution among team members ▶
- ☐ Students to explore and decide on UI/frontend framework and technologies ▶ ▶
- ☐ Students to research data storage fundamentals (storage hardware, RAID, servers, data center components) instead of databases ▶ ▶
- ☐ Students to evaluate single agent vs multi-agent system approach with pros and cons analysis ▶ ▶
- ☐ Students to explore agent communication mechanisms and orchestration strategies ▶
- ☐ Team to create WhatsApp group for ongoing communication ▶ ▶
- ☐ Team to set up recurring meetings once every three weeks ▶ ▶

Problem Statement Clarification

Core Challenge: Develop an agentic AI system to automate bug triaging across multiple disconnected bug tracking platforms (Jira, Bugzilla, Confluence, etc.) used by different teams within HPE's complex enterprise environment ▶ ▶

Real-World Context: HPE's hybrid cloud solutions are composite systems involving multiple components (servers, storage, networking, hypervisors, Kubernetes, cloud management platforms), each with separate teams and bug tracking systems. When customers log issues, the system must automatically

identify affected components, assign bugs to appropriate teams, and track resolution across all systems while presenting a unified view to customers ▶ ▶ ▶

Example Scenario: If a customer cannot create a VM in the private cloud platform, the failure could be caused by storage, server, or cloud management issues. The system should automatically assign the bug to all affected component teams, track each team's progress in their respective systems, and provide consolidated status updates ▶ ▶ ▶

Key Requirements:

- Fetch bug information from multiple heterogeneous data sources ▶
- Provide consolidated view across various bug tracking platforms ▶
- Track bugs to completion across multiple component teams ▶
- Support different personas (customers, executives, internal teams) with appropriate views ▶ ▶
- Gather all issues affecting a particular customer problem ▶
- Monitor SLA compliance and contract obligations ▶

Student Progress Review

Pain Points Identified:

- Manual triaging across disconnected systems ▶
- Lost tracking of bug states as teams use different platforms ▶
- Operational overhead from context switching between multiple repositories ▶
- Scattered data and manual mis-routing ▶

AI/ML Concepts Explored:

- Large Language Models (LLMs) and potential use of OpenAI, Claude, or open-source models ▶
- Tokenization and embeddings for text processing ▶
- Classification agents for severity assessment, team assignment, and timeline determination ▶
- CRUD agents and routing agents for automation ▶

- Tool calling and function calling for API interactions ▶

Database Technologies Explored (Note: This was a misunderstanding - team expected storage hardware fundamentals instead):

- Relational databases (PostgreSQL) for structured data and records ▶
- Graph databases (Neo4j) for tracking relationships and bug lineage across systems ▶
- In-memory cache (Redis) for fast querying and preventing duplicate bug processing ▶
- Message queues (Kafka) for handling concurrent bug submissions and agent failures ▶

Implementation Technologies:

- Python as main programming language for AI agents ▶
- Jupyter Lab or Google Colab for model testing and experimentation ▶
- Docker for running components in isolated containers ▶
- Kubernetes for scaling and managing multiple containers ▶
- Linux as base environment for system deployment ▶

Clarification on Data Storage Fundamentals

Misunderstanding Identified: Students researched databases when the team expected them to study storage hardware fundamentals ▶

Expected Topics:

- Block storage vs object storage ▶
- RAID configurations ▶
- Server and storage technologies ▶
- Data center components and architecture ▶
- Hybrid cloud infrastructure basics ▶

Architecture & Design Requirements

Reference Architecture Provided: HPE team shared a reference architecture as guidance, not a final design. Students must develop their own high-level architecture ▶ ▶ ▶

Key Components in Reference:

- UI frontend layer ▶
- Gateway for REST API access ▶
- Orchestrator module to coordinate multiple agents and connectors ▶
- Data plane for metadata and bug information storage ▶
- MCP servers for fetching information from external data sources ▶

Design Considerations:

- Apply system design patterns ▶
- Focus on scalability and performance ▶
- Consider how to map identified technologies to the problem statement ▶ ▶
- Create component diagrams showing building blocks and their relationships ▶

Multi-Agent System Considerations

Key Questions to Explore:

- Single agent vs multiple agents: evaluate pros and cons ▶ ▶
- Agent communication mechanisms and protocols ▶ ▶
- Agent monitoring and lifecycle management ▶
- Orchestrator or planning agent requirements ▶
- Level of human involvement in the workflow ▶

Determinism Requirements: System must provide deterministic responses (e.g., "bug state is open") rather than probabilistic ones (e.g., "could be open or closed") ▶

Guidance Provided: Multiple agents confirmed as necessary due to system complexity, but students should go through the evaluation process to understand why ▶ ▶ ▶

Frontend Considerations

Open Question: No progress shown on UI/frontend technology selection ▶

Required Exploration:

- Frontend framework selection ▶
- What to showcase in the prototype ▶
- How to present information for different personas ▶

Communication & Meeting Planning

- Team to create WhatsApp group for asynchronous communication and questions between meetings ▶ ▶
- Recurring meetings to be scheduled once every three weeks ▶ ▶
- Students can ask questions in WhatsApp group without waiting for meetings ▶

Project Timeline

Program runs until May, requiring students to plan work allocation and bandwidth accordingly ▶